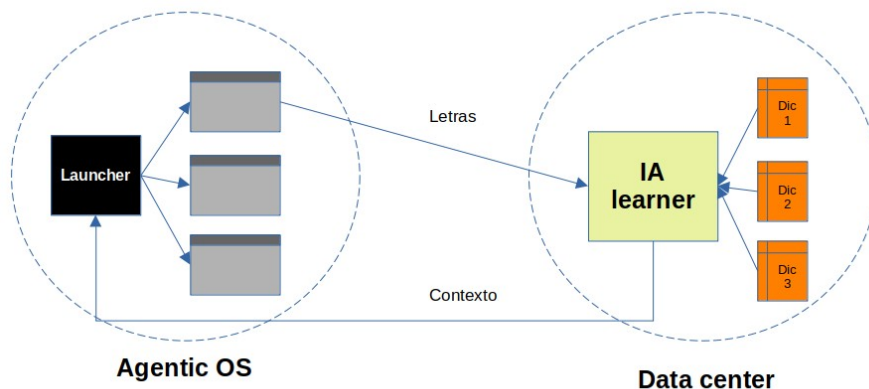


Agentic-OS

IBM secretamente quiere [re-ingresar](#) al negocio de los sistemas operativos, después de varias décadas de ausencia. para competir con Android y Windows. El factor diferenciador de su nuevo OS es el uso de [Agentes](#) de IA, pero para que dichos agentes hagan la vida del usuario más fácil, deben aprender sobre el contexto: que hace el usuario, sobre que trata la información que maneja, etc. IBM lo ha contratado secretamente a usted para que **diseñe e implemente** una arquitectura que permita responder a estas preguntas. El plan es embeber un [“backdoor”](#) en el API del sistema de ventanas (X11) de un sistema operativo POSIX compatible. Así cada vez que se cree una ventana y se escriba en ella, el texto sea enviado al sistema de IA de IBM para [entrenar](#) un agente de IA en base al [contexto del usuario](#). Para su primera entrega del sistema, IBM ha definido los siguientes requerimientos:

- 1) La ejecución y estado de los procesos debe ser monitoreado desde un “launcher” interactivo que se ejecute en una consola.
- 2) Se deben poder crear N programas graficos. El esqueleto del programa es provisto por IBM.
- 3) El programa provisto por IBM debe ser modificado que envíe las letras que el usuario pulsa en la ventana grafica al proceso remoto “IALearner” ubicado en el data center de la compañía.
- 4) Cada vez que “IALearner” recibe una palabra, por cada proceso remoto, la agrega hasta formar oraciones. Una oración termina con el carácter *return*. Esta tarea se realiza de forma concurrente y asíncrona conforme llegan los datos de cada ventana gráfica.
- 5) El algoritmo a entrenar usa la tecnica llamada [“bag of words”](#), donde cada linea de texto se convierte en un vector de frecuencias de palabras de un diccionario asociado a una clase de documento determinada.
- 6) Una vez que se conoce la clase de documento, “IALearner” va a inferir de que tipo de usuario se trata: *personal administrativo, directivo, o técnico* y lo mostrará en pantalla. Es será el *contexto del usuario*.

Abajo se muestra como interactuan algunos de los componentes mencionados (esto es solo con fines ilustrativos).



A fin de poder clasificar un ventana en un tipo de documento (la matriz de vectores de frecuencia de las lineas de texto) y el conjunto de documentos en un tipo de usuario, se pueden aplicar las siguientes reglas:

Si al menos tres palabras de un diccionario aparece en el documento, entonces el documento pertenece a la clase asociadas al diccionario. Si esto se cumple para más de un documento se escoge la clase que tenga la mayor suma de frecuencia de palabras. Los calculos se ejecutan cuando el proceso (ventana) se termina. Existen los siguientes diccionarios:

Correo electrónico	Artículo científico	Reporte
Thank,Please,Reg,ards,Meeting,Attached,Information ,Update,Schedule,Team, Project	Data ,Analysis,Results,Method,Study,Model,Research,System,Significant,Effect	System,Data,Network,Security,Application,Server,User,Performance,Service,Infrastructure.

Una vez terminados todos los procesos generados por el launcher, el IA learner debe determinar el tipo de usuario. Para ello usará la siguiente tabla de referencia, donde X puede ser la proporcion de documentos.

	Correo electrónico	Artículo científico	Reporte
Personal administrativo	X		
Personal tecnico	X		X
Profesor	X	X	
Estudiante		X	X

Entregables

- Un archivo ZIP con su nombre que contenga:
 1. Programa escrito en lenguaje C, puede usar las librerías [Pthreads](#) y/o [STL](#).
 2. El programa debe poderse compilar con gcc, debe venir un archivo README con instrucciones de compilación

3. Archivo PDF con un [diagrama de despliegue](#) y párrafo explicativo, máximo 2 carillas.

Rúbrica

Item	Ponderación
Los algoritmos seleccionados e implementados cumplen con los requerimientos de forma EFICIENTE	25
Los mecanismos de multi-programación seleccionados cumplen con los requerimientos de forma EFICIENTE	20
Los mecanismos de IPC seleccionados cumplen con los requerimientos de forma EFICIENTE	20
El documento de diseño esta BIEN escrito, tiene una estructura CORRECTA y se entiende la explicación provista acerca del funcionamiento de las partes más importantes de la solución propuesta	15
La codificación del programa cumple con criterios de programación DEFENSIVA .	20

Checklist

Item	X
El programa compila sin errores	
El programa es fácil de entender y tiene comentarios útiles	
El programa es más avanzado que un proyecto de fundamentos de programación que realice la misma tarea.	
El manejo de datos se hace con TDAs apropiadas según las operaciones a realizar sobre los datos.	
Las sentencias de repetición (while/for) emplean condiciones	
Las tareas del sistema se realizan en secciones definidas claramente en el programa	
Se minimiza el riesgo de inconsistencia de datos	
Los recursos se liberan cuando el programa termina	
No hay parámetros "quemados" en el programa	
Si un parámetro de entrada es incorrecto el programa no se termina	
Se validó que no haya condiciones de carrera en el programa propuesto	
No quedan procesos zombies o huérfanos	
El uso del CPU no se dispara de forma temporal o permanente	
Se explotan las técnicas discutidas en clase (multiprocesamiento, concurrencia, llamadas al sistema, etc).	